

Fragments

Q. What is fragment?

⇒ A fragment represents a reusable portion of app's UI.

⇒ A fragment defines and manages its own layout, has its own life cycle, and can handle its own input events. Fragments cannot live on their own. They must be hosted by an activity or another fragment.

⇒ it is also known as sub-activity.

⇒ There can be more than one fragment in an activity.

⇒ Fragments represent multiple screen inside one activity.

⇒ Fragment lifecycle is affected by activity life cycle.

Modularity - introduce modularity and reusability into activity's UI by letting to divide the UI into discrete chunks.

Q. How to Create a fragment?

(1) Set up environment:

Fragments require a dependency on AndroidX Fragment library.

In order to include this dependency we need to add the Google Maven repository in settings.gradle file.

(2) Create a fragment class:

```
class ExampleFragment extends Fragment
{
```

```
    public ExampleFragment()
    {
```

```
        super(R.layout.example_fragment);
    }
}
```

⇒ The fragment library also provides more specialized fragment base classes:

- DialogFragment
- PreferenceFragmentCompat

Add a fragment via XML:

⇒ To add a fragment to activity layout's XML, `FragmentContainerView` element is used.

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/frag1"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:name="com.example.ExampleFragment" />
```


⇒ The `android:name` attribute specifies the class name of the Fragment to instantiate.

⇒ When the activity layout is inflated, the specified fragment is instantiated, `onInflate()` is called on the newly instantiated fragment, and a `FragmentManager` is created to add the fragment to the `FragmentManager`.

Note:

The class attribute can be used instead of `android:name` as an alternative way to specify which Fragment to instantiate.

Q. What is fragment manager?

○ The `FragmentManager` is responsible for determining what state its fragment should be in and then moving them into that state.

○ `FragmentManager` is the class responsible for performing actions on App fragments, such as adding, removing, or replacing them and adding them to the back stack.

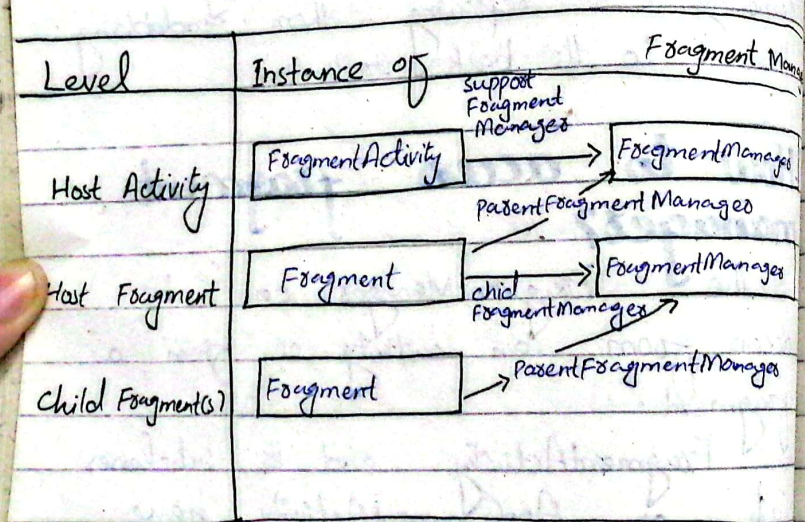
Q. How to access fragment manager?

⇒ The `FragmentManager` can be accessed from an activity or from a fragment.

⇒ `FragmentActivity` and its subclasses such as `AppCompatActivity`, have access to the `FragmentManager` through the `getSupportFragmentManager()` method.

⇒ Fragments can host one or more child fragments.

- Inside a fragment, it is possible to get a reference to the `FragmentManager` that manages the fragment's children through the `getChildFragmentManager()`.
- if need to access its host `FragmentManager`, use `getParentFragmentManager()`.



Each `FragmentManager` manages child fragments of the host.

Q. How `FragmentManager` manage backstack?

⇒ `FragmentManager` is responsible for managing the back stack of fragments. The back stack allows to navigate between fragments and maintain a history of fragment transactions.

⇒ Adding a fragment to the BackStack:

When a fragment transaction (e.g. adding, replacing, or removing a fragment), it can add it to the back stack by calling `addToBackStack(null)` on the `FragmentManager` object before commit the transaction.

- e.g.
- 1) `FragmentManager transaction = fragmentManager.beginTransaction();`
 - 2) `transaction.addToBackStack(null);`
 - 3) `transaction.commit();`
 - 4) `transaction.replace(R.id.fragment_container, new MyFragment());`

- This adds the transaction to the back stack with a null name.

2. Popping Fragments from the Back Stack:-

- To navigate back, we can use the `FragmentManager.popBackStack()` method.

- it is possible to pop the back stack one entry at a time or pop back to a specific fragment entry.

ex:

```
FragmentManager.popBackStack();
```

ex: back to a specific entry (by name or ID);

```
FragmentManager.popBackStack("fragment-name", FragmentManager.POP_BACK_STACK_INCLUSIVE);
```

3. Managing Back Stack entries:

→ To get information about the back stack entries we can use the methods like:

- `getBackStackEntryCount()`
- `getBackStackEntryAt()`

→ This can help to perform actions based on the state of the back stack.

Note: Proper back stack management is crucial for smooth app navigation.

Q. Explain Fragment Transactions

⇒ A fragment transaction is an operation that is used to add, replace, remove, or perform other changes to fragments within an activity.

⇒ it allows to dynamically add, remove, or swap fragments within the user interface of an app at runtime.

⇒ Some common operations performed with a fragment transaction are as follows:

(1) Adding a fragment

• Fragment transaction is used to add a new fragment to an activity's layout.

code:

```
1) FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
```

```
2) MyFragment fragment = new MyFragment();
```

```
(3) transaction.add(R.id.fragment_container, fragment);
```

```
(4) transaction.commit();
```

(2) Replacing a fragment

⇒ It is possible to replace one fragment with another using a `FragmentTransaction`. This is often used for navigation within an app.

code:

```
FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
```

```
MyFragment fragment = new MyFragment();  
transaction.replace(R.id.fragment_container, fragment);  
transaction.commit();
```

3) Removing a fragment

⇒ A fragment can be removed from layout.


```
FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
```

```
MyFragment fragmentToRemove = getSupportFragmentManager().findFragmentByTag("fragment-tag");
```

```
if (fragmentToRemove != null)
```

```
{
    transaction.remove(fragmentToRemove);
    transaction.commit();
}
```

4. Hiding and Showing Fragments:

○ Fragments can be hide and show without removing them completely which can be useful for managing the visibility.

Code:

```
FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
```

```
MyFragment fragmentToHide = getSupportFragmentManager().findFragmentByTag("fragment-tag");
```

```
if (fragmentToHide != null)
```

```
{
    transaction.hide(fragmentToHide);
    transaction.commit();
}
```

// To show the hidden fragment:-

```
FragmentTransaction showTransaction = getSupportFragmentManager().beginTransaction();
```

```
showTransaction.show(fragmentToHide);
showTransaction.commit();
```

5. Adding to the Back Stack:

⇒ Add a transaction to the back stack, allows users to navigate back to the previous state of app by pressing the back button.

Code:

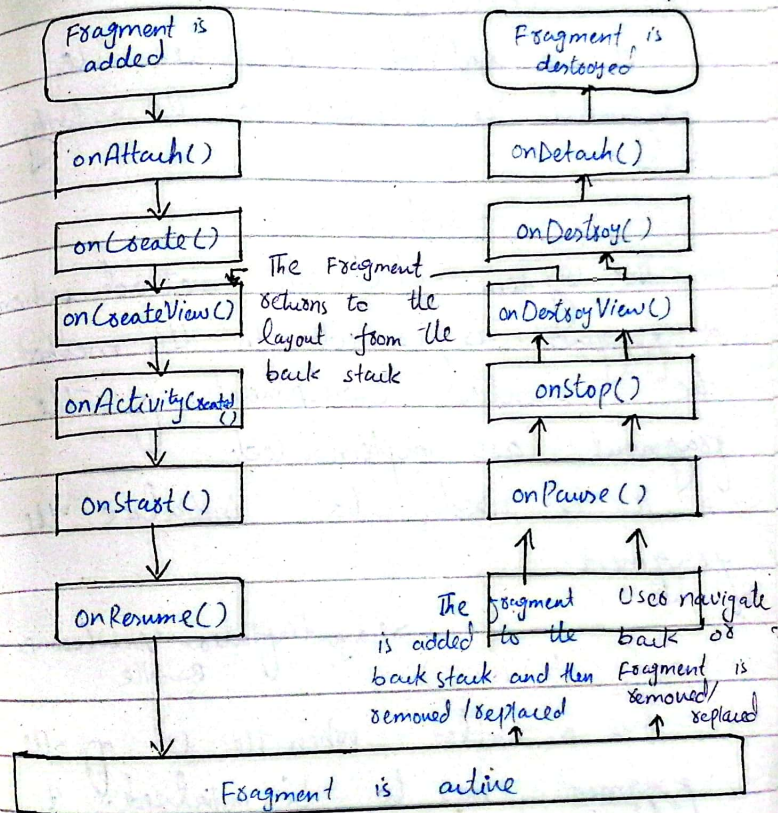
```
FragmentManager fragmentManager = getSupportFragmentManager();  
FragmentTransaction transaction = fragmentManager.beginTransaction();  
transaction.replace(R.id.fragment_container, fragment);  
transaction.addToBackStack(null);  
transaction.commit();
```

```
MyFragment fragment = new MyFragment();
```

```
transaction.replace(R.id.fragment_container, fragment);  
transaction.addToBackStack(null);  
transaction.commit();
```

Q-Explain fragment life cycle?

- The fragment lifecycle begins when it is attached to an activity.



Lifecycle methods for fragment

This is called when the fragment becomes active or interactive.

(1) onAttach(Activity):

- it is called once, when the fragment is attached to the activity.

(2) onCreate(Bundle):

- The system calls this method when a fragment is created. In this method the essential components of the fragment are implemented.
- it is used to initialize the fragment.

(3) onCreateView(): ^{→ LayoutInflater, ViewGroup, Bundle}

- This is called when the UI of the fragment has to be initialized. It is called when the fragment is first created and then when

the fragment returns back to the layout from the back stack. This method usually returns a View component but if the fragment does not have a UI, then you can return a null.

(4) onActivityCreated(Bundle):

- This method is called when the host activity is created. By this time, we can access the fragment's view using the `findViewById()` method.
- it is invoked after the completion of `onCreate()` method.

(5) onStart():

- This method is called when the fragment becomes visible on the device screen.

(6) onResume():

- it makes the fragment interactive.

onPause():

- it is called when the fragment is no longer interactive and the user is about to leave the fragment.

onStop():

- This method is called when the fragment is no longer visible.

onDestroyView():

- This is called when the fragment is to be destroyed. Here you can do the clean up the resources before the fragment is destroyed.

onDestroy():

- This is called for the final clean up of fragment's state.

onDetach():

- it is called just before the fragment is detached from the host activity.

Q- What are main uses of fragments?

1) Modularity:

if a single activity is having too many functional components, it is better to divide it into independent fragments, hence making the code more organized and easier to maintain.

2) Reusability.

3) Adaptability

Example:

Activity-main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    ... >
```

```
<fragment
    android:id="@+id/fragment1"
    android:layout_width="0px"
    android:layout_height="match-parent"
    android:layout_weight="1"
    android:name="example.com.fragment.fragments2"
/>
```

```
<fragment
    android:id="@+id/fragment2"
    android:name="..."
    android:layout_width="0px"
    android:layout_height="match-parent"
    android:layout_weight="1"
/>
```

```
</LinearLayout>
```

fragment-fragments.xml

<FrameLayout

>

<TextView

```
    android:layout_width="match-parent"
    android:layout_height="match-parent"
    android:text="@string/hello_blank_fragment"/>
```

</FrameLayout>

fragments.java

```
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.os.Bundle;
```

```
public class Fragment1 extends Fragment
{
```

@Override


```

public void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
}

```

@Override

```

public View onCreateView (LayoutInflater
    inflater, ViewGroup container, Bundle
    savedInstanceState)

```

```

{

```

// Inflate the layout for this fragment

```

return inflater.inflate (R.layout.fragment
    fragment, container, false);
}

```

```

}

```


Q. How to move from one fragment to another fragment?

```
<?xml version="1.0" encoding="utf-8">
```

```
<LinearLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
xmlns:tools="http://schemas.android.com/tools"
```

```
android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
```

```
android:orientation="vertical"
```

```
tools:context=".MainActivity">
```

```
<LinearLayout
```

```
android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
```

```
>
```

```
<Button
```

```
android:text="Frage A"
```

```
.....
```

```
/>
```

```
<Button
```

```
android:text="Frage B"
```

```
....
```

```
/>
```

```
<Button
```

```
android:text="Frage C"
```

```
....
```

```
/>
```

```
</LinearLayout>
```

```
<FrameLayout
```

```
android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
```

```
android:id="@+id/Container"/>
```

```
</LinearLayout>
```

FragmentA

```
import android.os.Bundle;
```

```
import androidx.fragment.app.Fragment;
```

```
import android.view.LayoutInflater;
```

```
import android.view.View;
```

```
import android.view.ViewGroup;
```

```
import android.widget.TextView;
```



```

Public class AFxagment extends Fragment
{

```

```

    Public AFxagment ()
    {

```

```

        // Constructors

```

```

    }

```

```

    @Override

```

```

    Public View onCreateView (LayoutInflater
    inflater, ViewGroup container, Bundle
    savedInstanceState)
    {

```

```

        // Inflate the layout

```

```

        View view = inflater.inflate (R.layout.fragment_
        container, false);
        return view;
    }

```

```

}

```

MainActivity.java

Package

com. cs2415.fragmentproject;

```

import androidx.appcompat.app.AppCompatActivity;

```

```

import androidx.fragment.app.Fragment;

```

```

import androidx.fragment.app. fragment app.
    FragmentManager;

```

```

import androidx.fragment.app.FragmentManager;

```

```

import android.os.Bundle;

```

```

import android.view.View;

```

```

import android.widget.Button;

```

```

Public class MainActivity extends
    AppCompatActivity

```

```

{

```

```

    Button btnfxaA, btnfxbB, btnfxcC;

```

```

    @Override

```

```

    Protected void onCreate (Bundle savedInstanceState)
    {

```

```

        super.onCreate (savedInstanceState)

```

```

        setContentView (R.layout.activity_main);

```

```

        btnfxaA = findViewById (R.id. btnfxaA);

```

```

        ;
        ;
        ;

```



```
loadFrags ( new BFrags ( ), 0 ); //  
default frags
```

```
btnA: setOnClickListener ( new View.  
OnClickListener () {  
    @Override  
    public void onClick ( View view )  
    {  
        loadFrags ( new AFrags ( ), 1 );  
    }  
} );
```

// do same for B and C

```
public void loadFrags ( Fragment frags  
int flag )  
{
```

```
    FragmentManager fm = getSupportFragmentManager ();
```

```
    FragmentTransaction ft = fm.beginTransaction ();
```

```
    if ( flag == 0 )
```

```
        ft.add ( R.id.container, frags );
```

```
    else
```

```
        ft.replace ( R.id.container, frags );  
        ft.commit ();
```